Reto Técnico - Full Stack Developer

Descripción del Reto

Bienvenido/a a nuestra prueba técnica. Tu objetivo es desarrollar una **aplicación web completa** para la gestión de productos (CRUD) utilizando **Ruby on Rails** como backend y **React** como frontend.

Este reto nos permitirá evaluar tus habilidades técnicas, capacidad de resolución de problemas, y tu familiaridad con las mejores prácticas de desarrollo.

🎯Objetivo

Crear un sistema de gestión de productos que permita:

- ✅ Listar productos con paginación
- ✅ Crear nuevos productos
- ✅ Editar productos existentes
- ✅ Eliminar productos
- ✅ Buscar productos por nombre
- ✅ Filtrar productos por estado (activo/inactivo)

🛠 Stack Tecnológico Requerido

Backend

- **Ruby on Rails**
- **PostgreSQL** como base de datos
- **API REST** (modo API)

Frontend

- **React**
- **TypeScript** (obligatorio)
- Framework CSS de tu elección (TailwindCSS, Material-UI, etc.)

Testing

- **RSpec** para tests unitarios del backend (obligatorio)
- Tests de frontend (opcional, pero valorado)

📦 Requerimientos Funcionales

1. Backend - API REST

Modelo de Datos: Product

El modelo `Product` debe contener los siguientes campos:

Campo	Tipo	Requerido	Validaciones
`name`	String	Sí	Longitud mínima: 3, máxima: 100
`description`	Text	No	Longitud máxima: 1000
`price`	Decimal	Sí	Debe ser mayor a 0
`stock`	Integer	Sí	No puede ser negativo
`sku`	String	Sí	Único, solo letras mayúsculas y números
`active`	Boolean	Sí	Default: true
`created_at`	DateTime	Automático	-
`updated_at`	DateTime	Automático	-

Endpoints Requeridos

GET /api/v1/products # Listar productos (paginado)

GET /api/v1/products/:id # Obtener un producto

POST /api/v1/products # Crear producto

PUT /api/v1/products/:id # Actualizar producto

DELETE /api/v1/products/:id # Eliminar producto

Características del API

- **Paginación**: Los listados deben estar paginados (10 items por página por defecto)
- **Búsqueda**: Implementar búsqueda por nombre de producto
- **Filtros**: Permitir filtrar por estado (activo/inactivo)
- **Validaciones**: Validar todos los datos antes de guardar
- **Manejo de errores**: Respuestas apropiadas para errores (404, 422, 500)
- **CORS**: Configurar CORS para permitir peticiones del frontend

2. Frontend - React

Características Requeridas

1. **Listado de Productos**

    - Mostrar productos en tarjetas o tabla
    - Paginación funcional
    - Barra de búsqueda
    - Filtro por estado (activo/inactivo)
    - Loading states

2. **Formulario de Producto**

    - Modal o página separada para crear/editar
    - Validación de campos en tiempo real
    - Mensajes de error claros
    - Confirmación antes de guardar

3. **Eliminar Producto**

    - Confirmación antes de eliminar
    - Feedback visual (toast/notification)

4. **Diseño**

    - Responsive (mobile, tablet, desktop)
    - UI limpia y moderna
    - Estados de loading
    - Manejo de errores visualmente



Validaciones del Frontend

- Nombre: mínimo 3 caracteres, máximo 100
- SKU: solo letras mayúsculas, números y guiones
- Precio: mayor a 0, formato decimal
- Stock: número entero no negativo
- Descripción: máximo 1000 caracteres (opcional)

Backend (Obligatorio)

Debes incluir tests para:

- ✅ **Validaciones del modelo**: Probar todas las validaciones
- ✅ **Scopes**: Si implementas scopes, probar que funcionen
- ✅ **API Endpoints**: Tests de integración para todos los endpoints
- ✅ **Casos edge**: Probar casos límite y errores

Criterios de Evaluación

Tu solución será evaluada en base a:

1. Funcionalidad (30%)

- ✅ Todas las funcionalidades CRUD funcionan correctamente
- ✅ Validaciones funcionan en backend y frontend
- ✅ Búsqueda y filtros operativos
- ✅ Paginación funcional

2. Código Backend (25%)

- ✅ Estructura y organización del proyecto
- ✅ Uso correcto de Rails conventions
- ✅ Validaciones a nivel de modelo
- ✅ Manejo apropiado de errores
- ✅ Tests unitarios completos y que pasen
- ✅ Seeds funcionales

3. Código Frontend (25%)

- ✅ Componentes bien estructurados
- ✅ Uso correcto de TypeScript
- ✅ Manejo de estado apropiado
- ✅ Validaciones en formularios
- ✅ UX intuitiva
- ✅ Código limpio y mantenible

4. Buenas Prácticas (20%)

- ✅ Código limpio y legible
- ✅ Nombres descriptivos
- ✅ Separación de responsabilidades
- ✅ Commits atómicos y descriptivos
- ✅ README con instrucciones claras
- ✅ Variables de entorno para configuración

Plus Points (Bonus)

- 💥 Implementación de tests en frontend
- 💥 Uso de React Query o similar para gestión de estado

- 🌟 Dockerización de la aplicación
- 🌟 CI/CD configurado
- 🌟 Deploy funcional (Heroku, Railway, Vercel, etc.)
- 🌟 Documentación de API (Swagger/OpenAPI)
- 🌟 Soft deletes en lugar de hard deletes
- 🌟 Auditoría de cambios (quién y cuándo modificó)

📝 Instrucciones de Entrega

1. Repositorio

- Crea un **repositorio público** en GitHub
- Incluye dos carpetas: `backend/` y `frontend/`
- Commits claros y descriptivos
- No incluir `.env` ni credenciales

2. Documentación Requerida

README.md Principal

Decisiones Técnicas

Explica decisiones importantes que tomaste y por qué

Mejoras Futuras

Qué agregarías con más tiempo

Preguntas Frecuentes

¿Puedo usar librerías adicionales?

Sí, siempre que estén justificadas. Documenta en el README por qué las elegiste.

¿Necesito desplegar la aplicación?

No es obligatorio, pero es muy valorado. Puedes usar servicios gratuitos como Railway, Render, o Vercel.

¿Qué hago si no termino todo?

Prioriza la funcionalidad core y documenta qué falta y cómo lo implementarías. Es mejor entregar algo funcional parcial que algo incompleto que no corre.

¿Puedo usar un framework CSS?

Sí, usa el que prefieras: TailwindCSS, Material-UI, Bootstrap, Chakra UI, etc.

¿Necesito autenticación?

No, este reto no requiere autenticación. Enfócate en el CRUD.

¿Puedo usar generadores de Rails?

Sí, pero asegúrate de entender el código generado y personalizarlo según necesites.

🌟 Lo Que Nos Gustaría Ver

Más allá de completar el reto, nos impresionaría ver:

1. Atención al detalle UX pulida, mensajes de error útiles, validaciones consistentes

2. Código profesional: Limpio, organizado, comentarios donde agregan valor

3. Pensamiento crítico: Decisiones técnicas bien fundamentadas

4. Proactividad: Features adicionales que mejoren la experiencia

5. Comunicación: README claro que demuestra que puedes explicar tu trabajo

¡Mucha suerte! Esperamos ver tu creatividad y habilidades técnicas en la pole position.